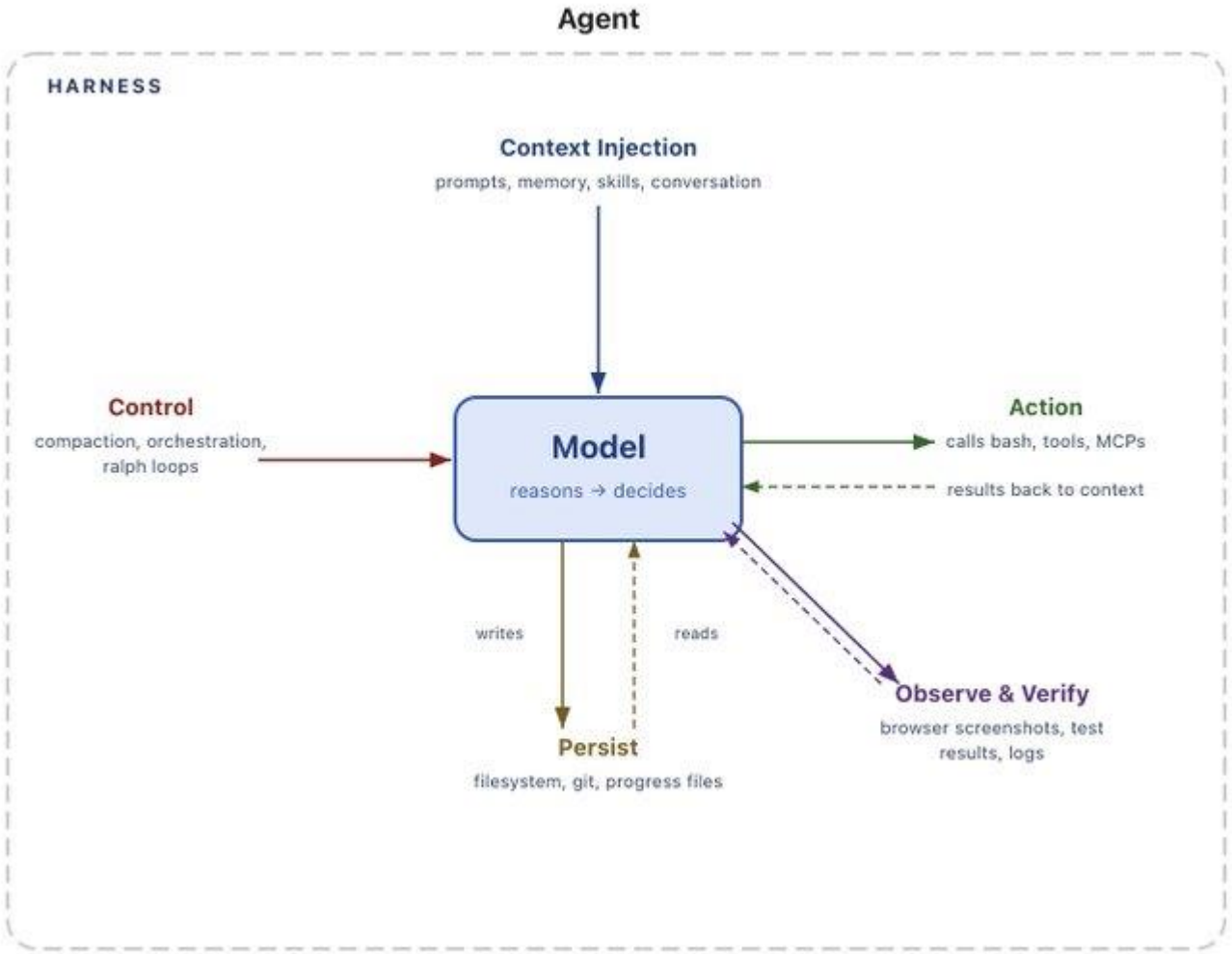
Agents and Harnesses

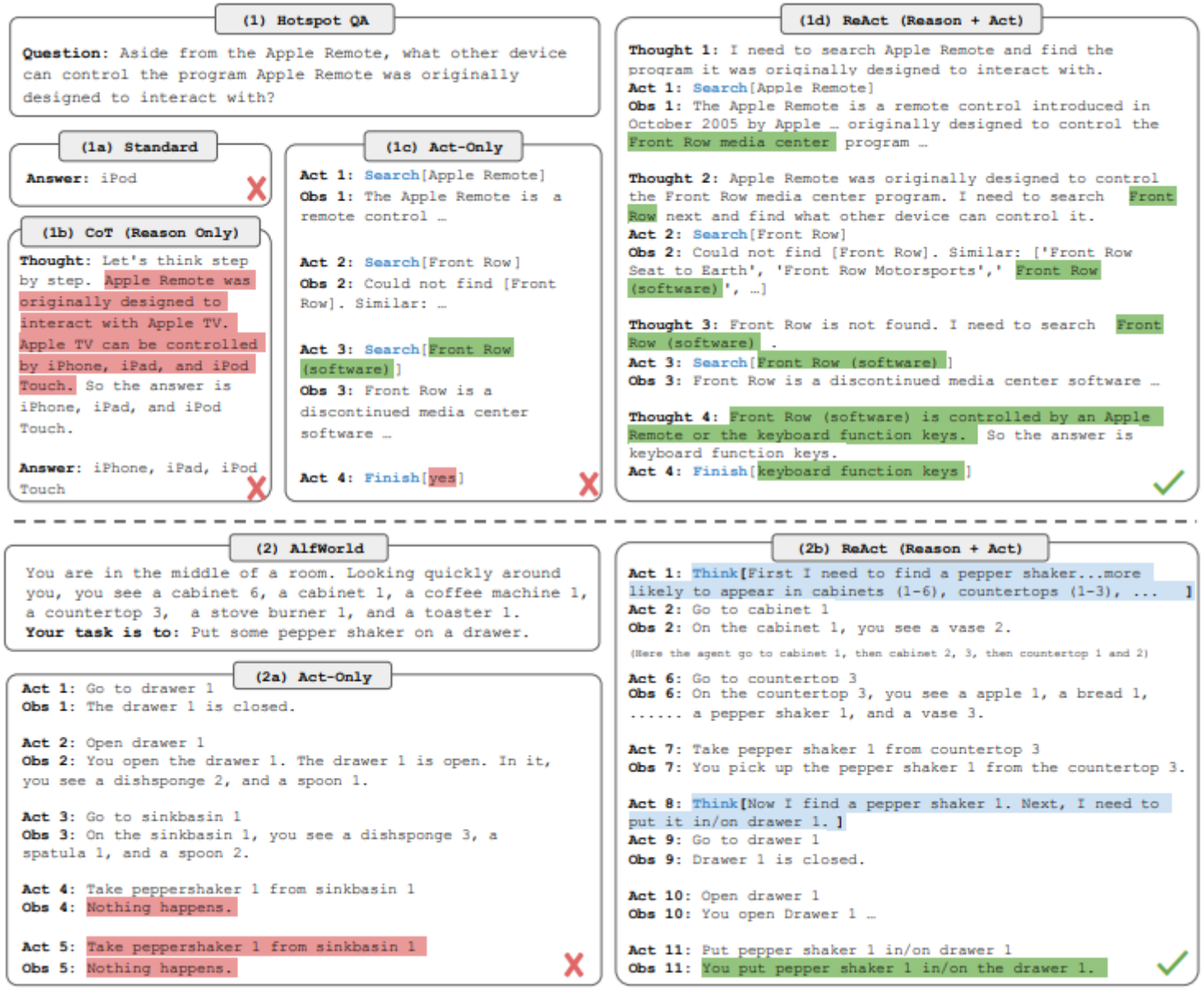
Core terms

Agent = LLM + harness



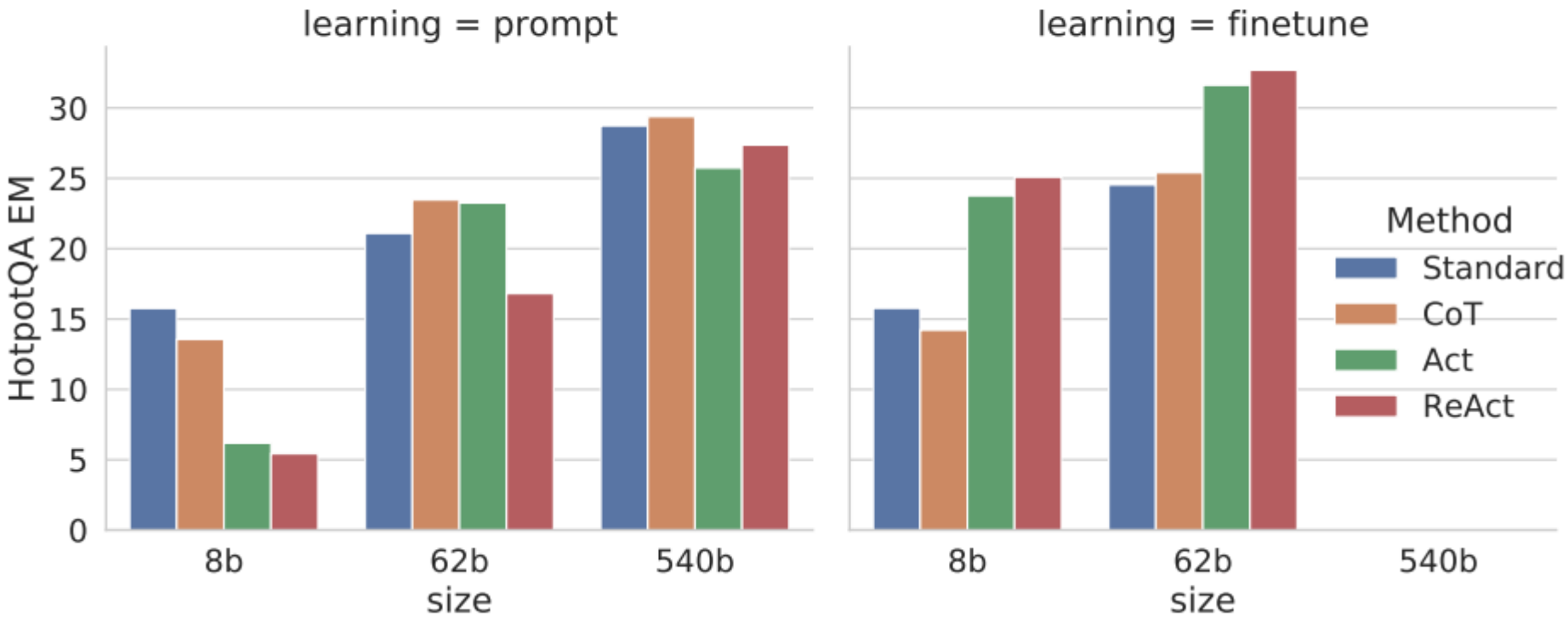
Anthropic: Building effective agents

ReAct



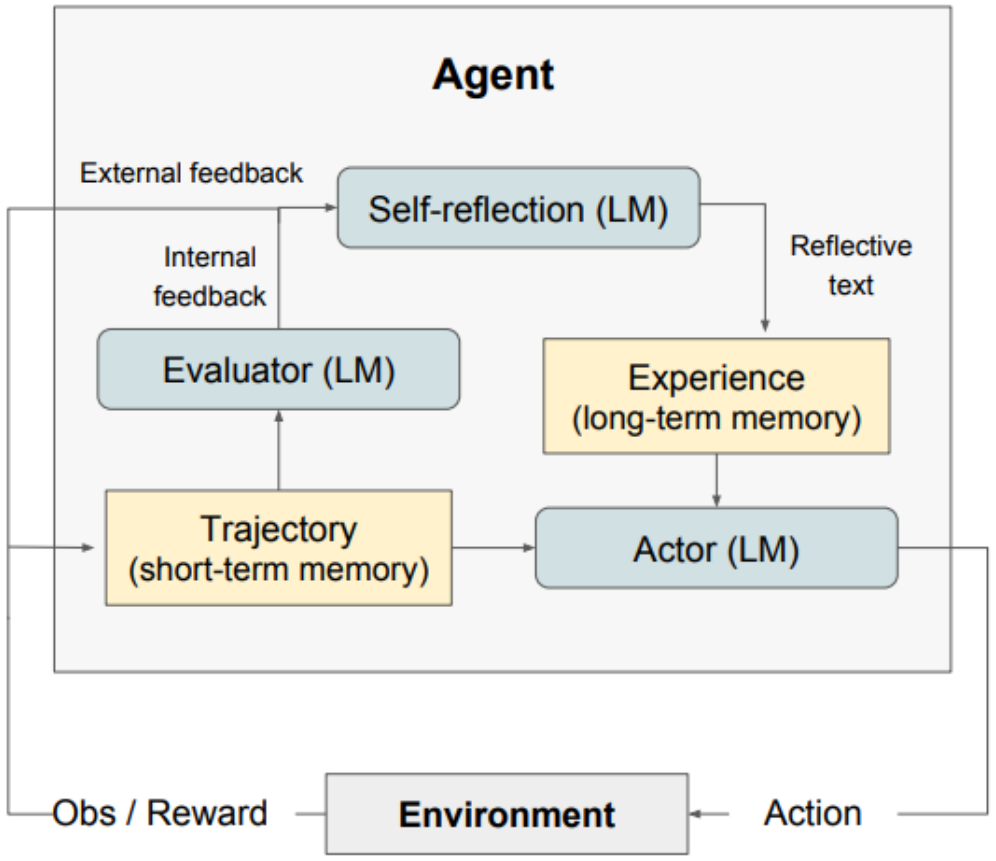
статья

ReAct: Finetunig



[статья]

Reflexion



Algorithm 1 Reinforcement via self-reflection

Initialize Actor, Evaluator, Self-Reflection: M_a, M_e, M_{sr}
Initialize policy $\pi_\theta(a_i|s_i), \theta = \{M_a, mem\}$
Generate initial trajectory using π_θ
Evaluate τ_0 using M_e
Generate initial self-reflection sr_0 using M_{sr}
Set $mem \leftarrow [sr_0]$
Set $t = 0$
while M_e not pass or $t <$ max trials **do**
    Generate $\tau_t = [a_0, o_0, \ldots a_i, o_i]$ using π_θ
    Evaluate τ_t using M_e
    Generate self-reflection sr_t using M_{sr}
    Append sr_t to mem
    Increment t
end while
return

Статья

Reflexion

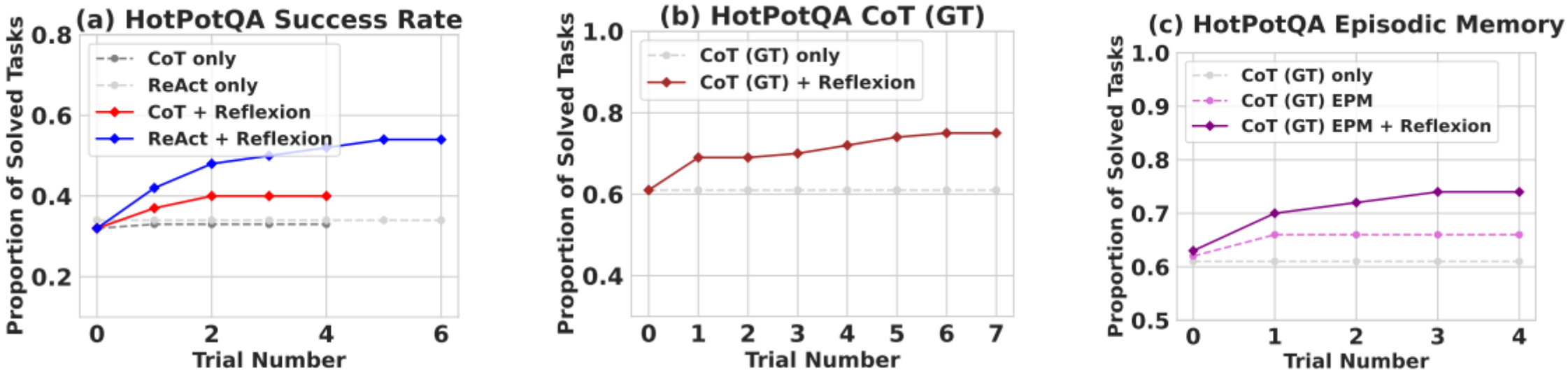
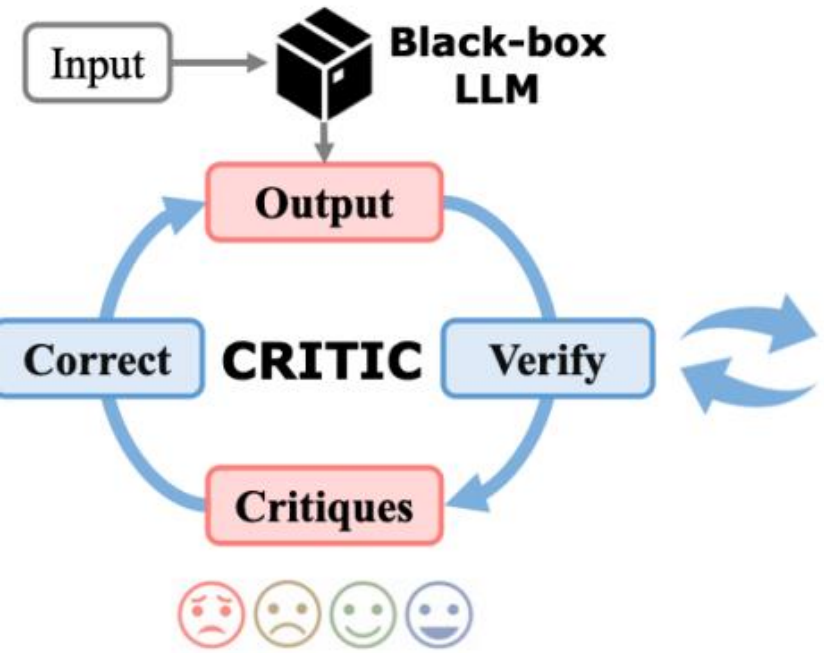


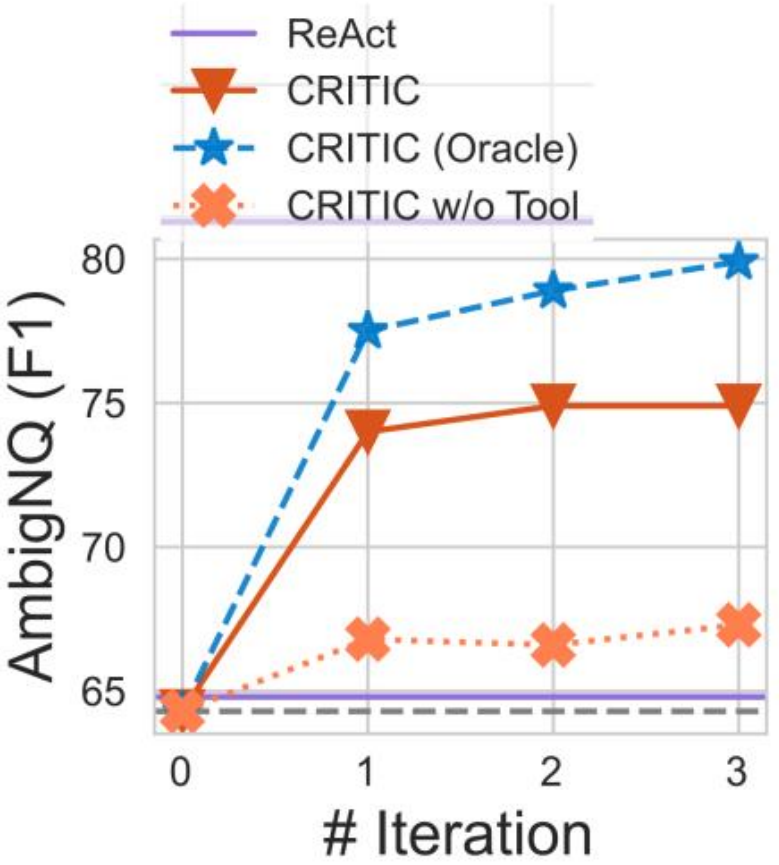
Figure 4: Chain-of-Thought (CoT) and ReAct. Reflexion improves search, information retrieval, and reasoning capabilities on 100 HotPotQA questions. (a) Reflexion ReAct vs Reflexion CoT (b) Reflexion CoT (GT) for reasoning only (c) Reflexion vs episodic memory ablation.

Статья

CRITIC


Input
Black-box LLM
Output
Correct
CRITIC
Verify
Critiques


External Tools
Knowledge Base
Code Interpreter
Text APIs
Perspective
Wikipedia
Calculator
Search Engine
Google


ReAct
CRITIC
CRITIC (Oracle)
CRITIC w/o Tool
AmbigNQ (F1)
80
75
70
65
0
1
2
3
Iteration

CodeAct

JSON Actions

Instruction:

Determine the most cost-effective country to purchase the smartphone model “CodeAct1”. The countries to consider are the USA, Japan, Germany, and India.

LLM Agent using [**Text/JSON**] as Action

Think I should calculate the phone price in USD for each country, then find the most cost-effective country.

Action
Text: lookup_rates, Germany
JSON: {"tool": "lookup_rates", "country": "Germany"}

Environment 1.1, 0.19

Action
Text: lookup_phone_price, CodeAct 1, Germany
JSON: {"tool": "lookup_phone_price", "model": "CodeAct 1", "country": "Germany"}

Environment 700

Action
Text: convert_and_tax, 700, 1.1, 0.19
JSON: {"tool": "convert_and_tax", "price": 700, "exchange_rate": 1.1, "tax_rate": 0.19}

Environment 916.3

[... interactions omitted (look up shipping cost and calculate final price) ...]

Action
Text: lookup_rates, Japan
JSON: {"tool": "lookup_rates", "country": "Japan"}

[... interactions omitted (calculate final price for all other countries)...]

Response The most cost-effective country to purchase the smartphone model is Japan with price 904.00 in USD.

Available APIs

[1] lookup_rates(country: str) -> (float, float)
[2] convert_and_tax(price: float, exchange_rate: float, tax_rate: float) -> float
[3] estimate_final_price(converted_price: float, shipping_cost: float) -> float
[4] lookup_phone_price(model: str, country: str) -> float
[5] estimate_shipping_cost(destination_country: str) -> float

Paper link

Code Actions

Instruction:

Determine the most cost-effective country to purchase the smartphone model “CodeAct 1”. The countries to consider are the USA, Japan, Germany, and India.

CodeAct: LLM Agent using [**Code**] as Action

Think

I should calculate the phone price in USD for each country, then find the most cost-effective country.

Action

```
countries = ['USA', 'Japan', 'Germany', 'India']
final_prices = {}

for country in countries:
    exchange_rate, tax_rate = lookup_rates(country)
    local_price = lookup_phone_price("CodeAct 1", country)
    converted_price = convert_and_tax(
      local_price, exchange_rate, tax_rate
    )
    shipping_cost = estimate_shipping_cost(country)
    final_price = estimate_final_price(converted_price, shipping_cost)
    final_prices[country] = final_price

most_cost_effective_country = min(final_prices, key=final_prices.get)
most_cost_effective_price = final_prices[most_cost_effective_country]
print(most_cost_effective_country, most_cost_effective_price)
```

Control & Data Flow of Code Simplifies Complex Operations

Re-use `min` Function from Existing Software Infrastructures (Python library)

Environment

```
Japan 904.0
```

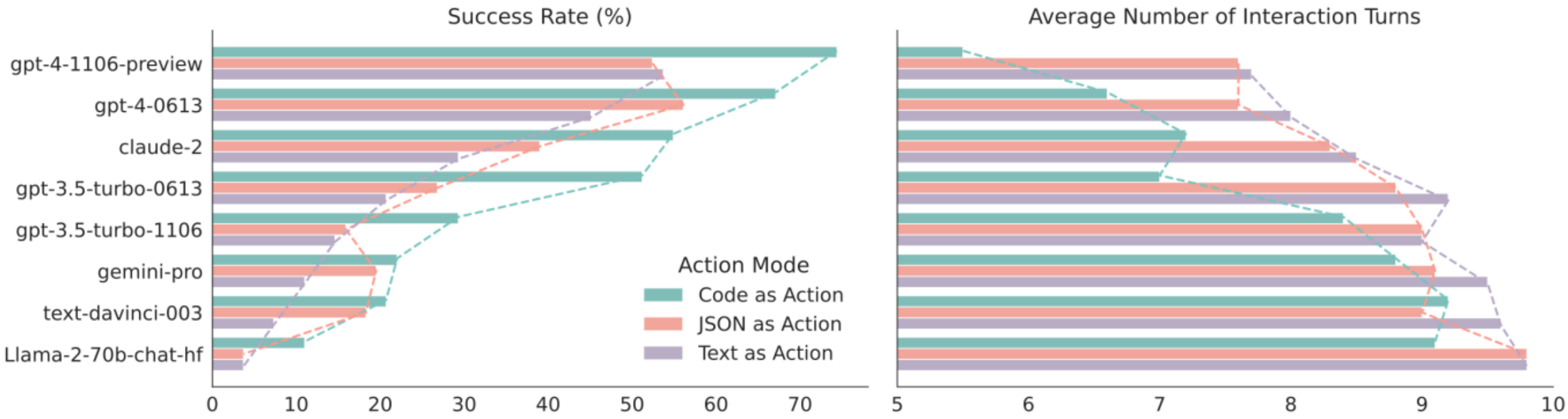
Response

The most cost-effective country to purchase the smartphone model is Japan with price **904.00** in USD.

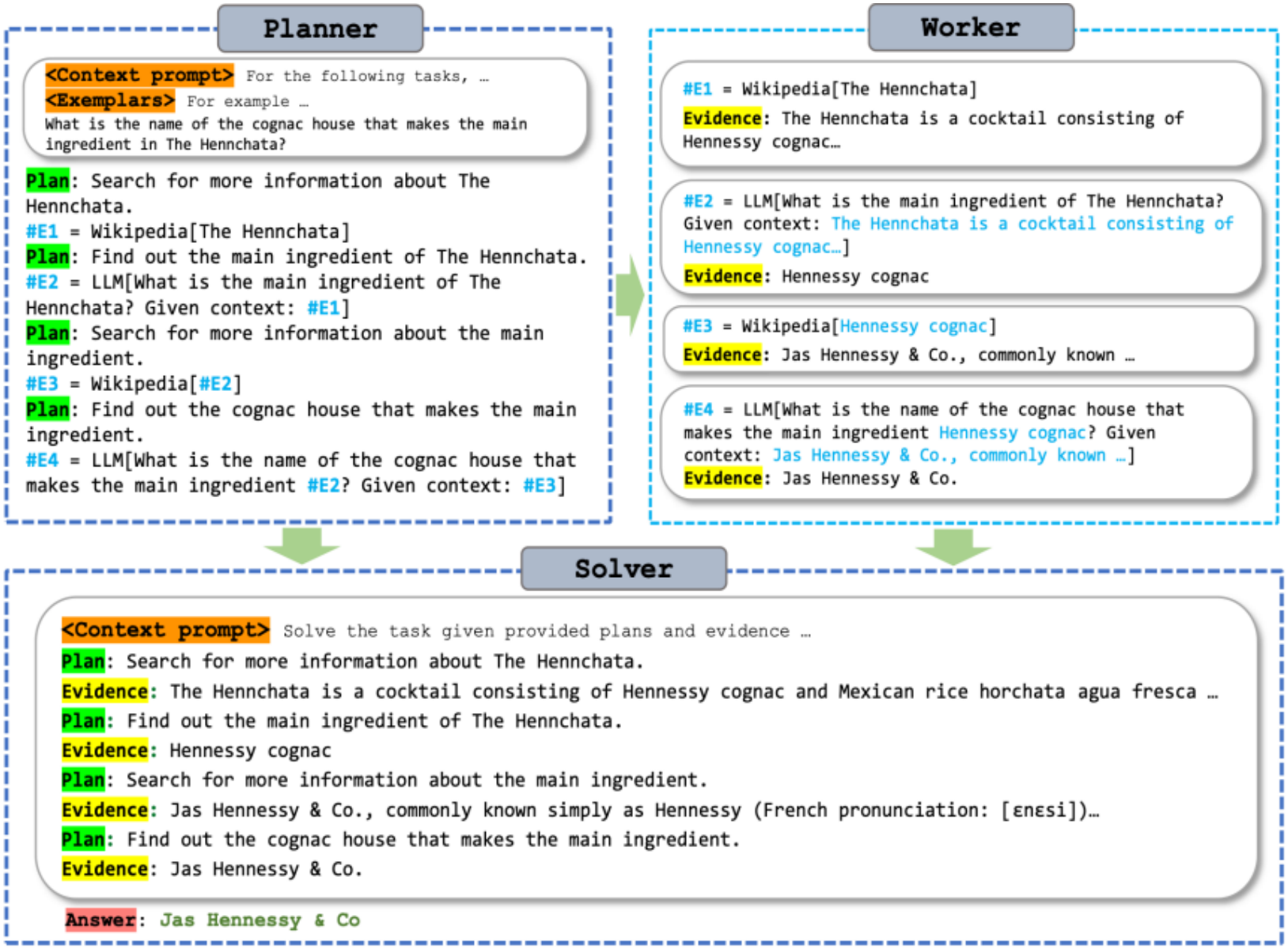
Available APIs

[1] lookup_rates(country: str) -> (float, float)
[2] convert_and_tax(price: float, exchange_rate: float, tax_rate: float) -> float
[3] estimate_final_price(converted_price: float, shipping_cost: float) -> float
[4] lookup_phone_price(model: str, country: str) -> float
[5] estimate_shipping_cost(destination_country: str) -> float

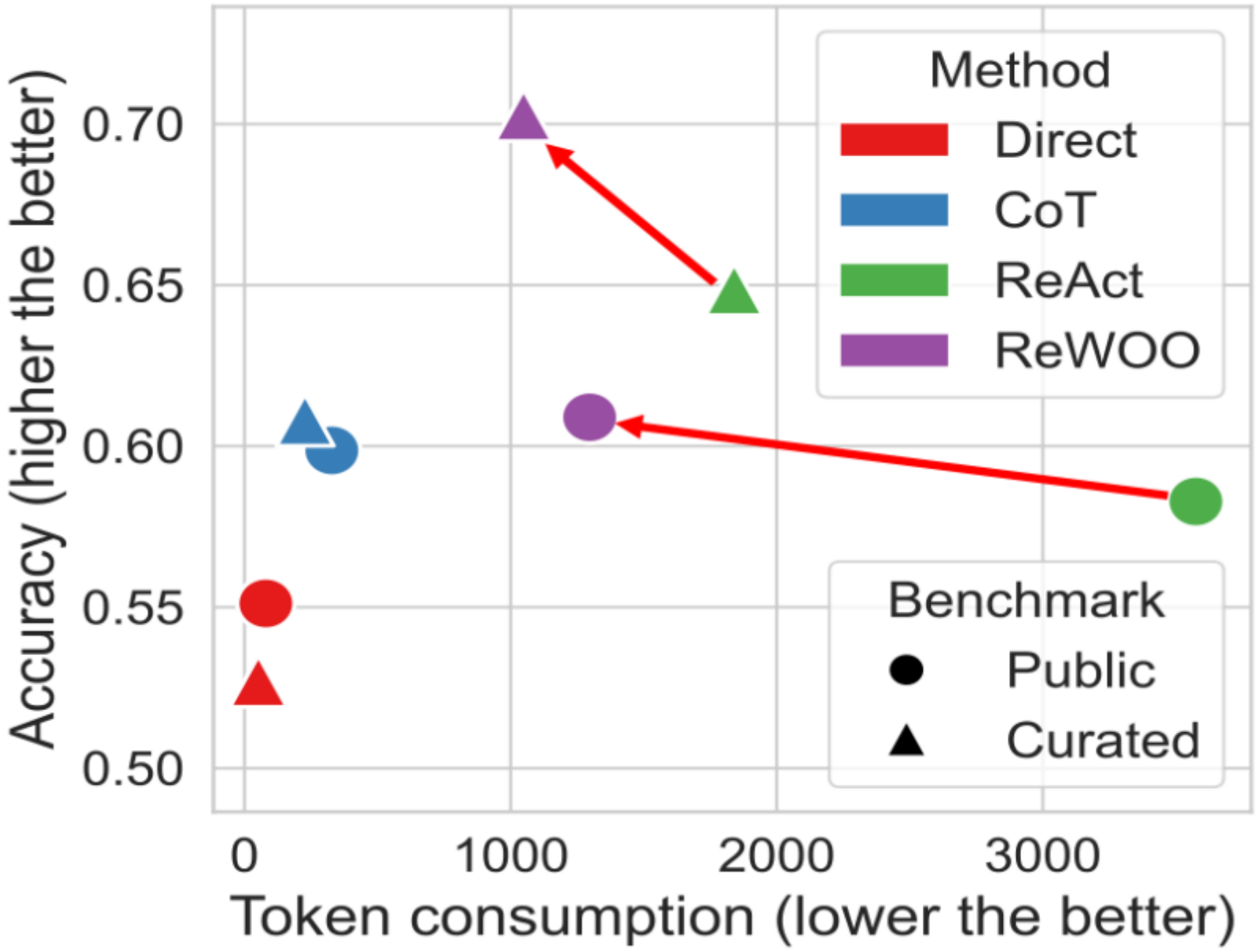
Code as action increases accuracy



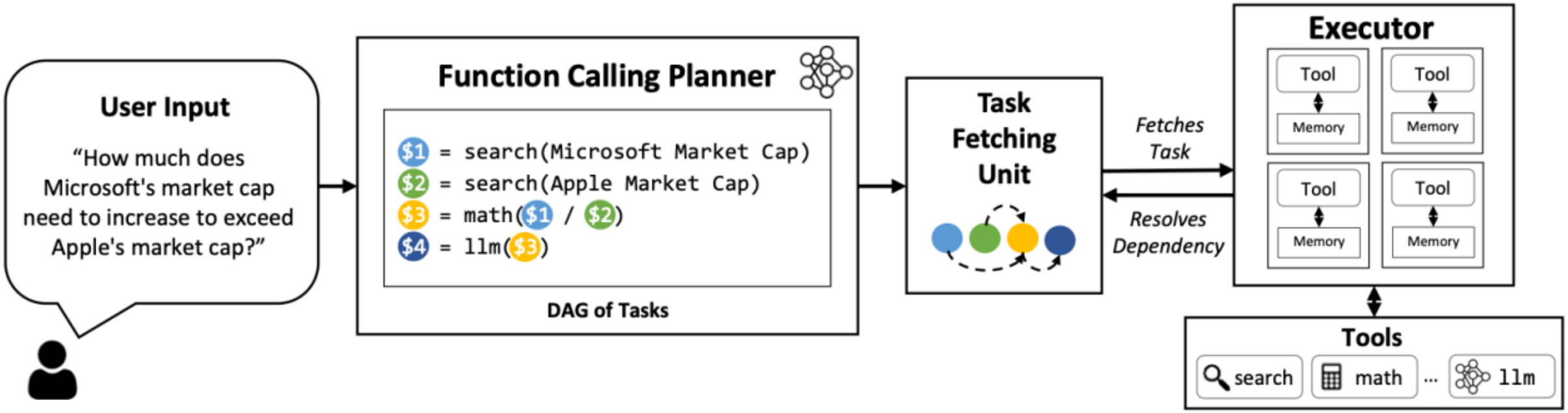
ReWOO


Planner
<Context prompt> For the following tasks, …
<Exemplars> For example …
What is the name of the cognac house that makes the main ingredient in The Hennchata?
Plan: Search for more information about The Hennchata.
#E1 = Wikipedia[The Hennchata]
Plan: Find out the main ingredient of The Hennchata.
#E2 = LLM[What is the main ingredient of The Hennchata? Given context: #E1]
Plan: Search for more information about the main ingredient.
#E3 = Wikipedia[#E2]
Plan: Find out the cognac house that makes the main ingredient.
#E4 = LLM[What is the name of the cognac house that makes the main ingredient #E2? Given context: #E3]
Worker
#E1 = Wikipedia[The Hennchata]
Evidence: The Hennchata is a cocktail consisting of Hennessy cognac…
#E2 = LLM[What is the main ingredient of The Hennchata? Given context: The Hennchata is a cocktail consisting of Hennessy cognac…]
Evidence: Hennessy cognac
#E3 = Wikipedia[Hennessy cognac]
Evidence: Jas Hennessy & Co., commonly known …
#E4 = LLM[What is the name of the cognac house that makes the main ingredient Hennessy cognac? Given context: Jas Hennessy & Co., commonly known …]
Evidence: Jas Hennessy & Co.
Solver
<Context prompt> Solve the task given provided plans and evidence …
Plan: Search for more information about The Hennchata.
Evidence: The Hennchata is a cocktail consisting of Hennessy cognac and Mexican rice horchata agua fresca …
Plan: Find out the main ingredient of The Hennchata.
Evidence: Hennessy cognac
Plan: Search for more information about the main ingredient.
Evidence: Jas Hennessy & Co., commonly known simply as Hennessy (French pronunciation: [ɛnɛsi])…
Plan: Find out the cognac house that makes the main ingredient.
Evidence: Jas Hennessy & Co.
Answer: Jas Hennessy & Co

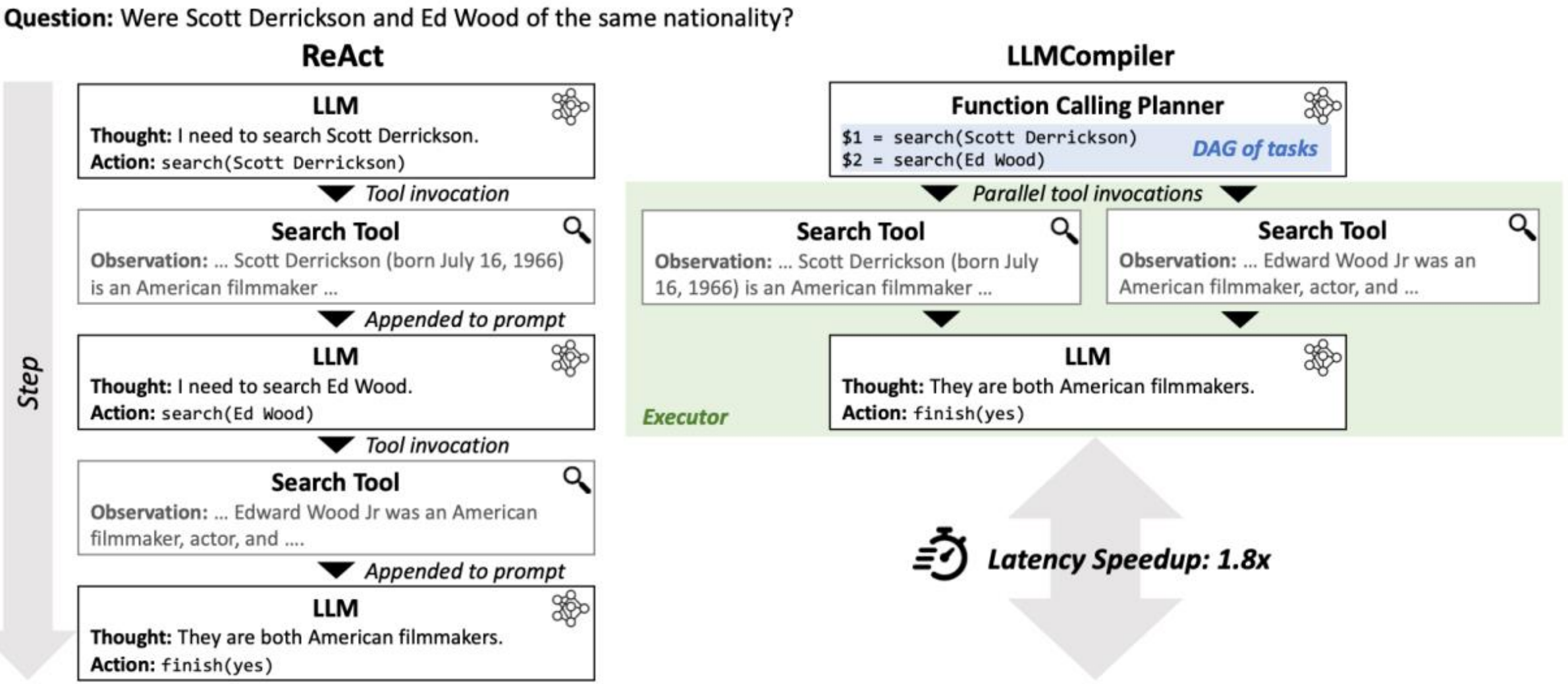
ReWOO


Method
Direct
CoT
ReAct
ReWOO
Benchmark
Public
Curated
Accuracy (higher the better)
0.70
0.65
0.60
0.55
0.50
0
1000
2000
3000
Token consumption (lower the better)

LLMCompiler



LLMCompiler

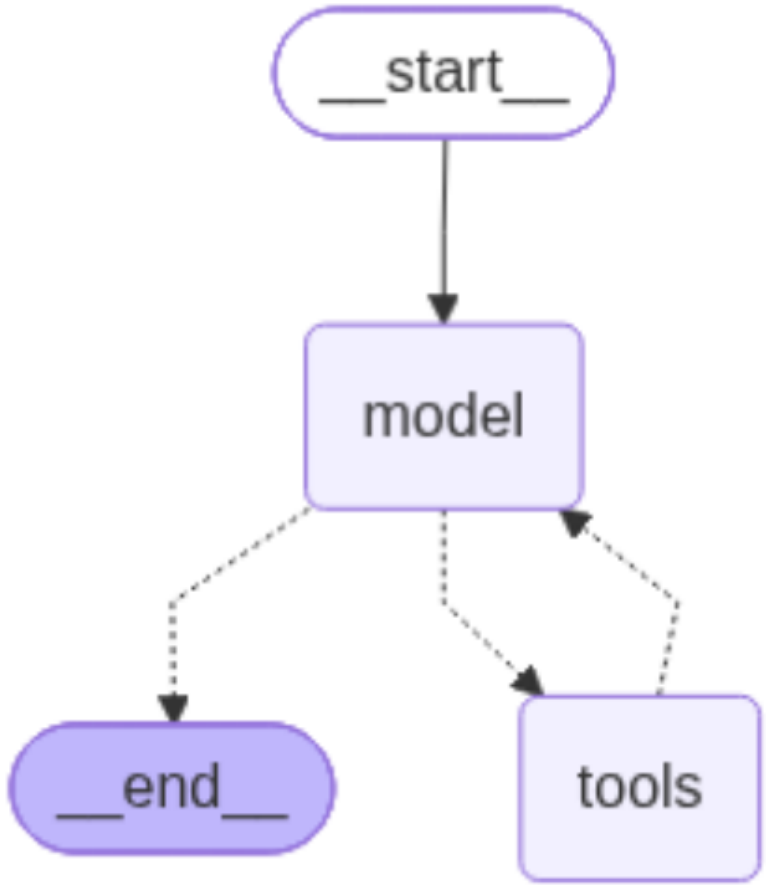


LLMCompiler

Benchmark	Method	GPT (Closed-source)			LLaMA-2 70B (Open-source)		
		Accuracy (%)	Latency (s)	Speedup	Accuracy (%)	Latency (s)	Speedup
HotpotQA	ReAct	61.52	-	-	54.74	-	-
	ReAct†	62.47	7.12	1.00×	54.40	13.44	1.00×
	OAI Parallel Function	62.05	4.42	1.61×	-	-	-
	LLMCompiler	62.00	**3.95**	**1.80**×	57.83	**9.58**	**1.40**×
Movie Rec.	ReAct	68.60	-	-	70.00	-	-
	ReAct†	72.47	20.47	1.00×	70.60	33.37	1.00×
	OAI Parallel Function	77.00	7.42	2.76×	-	-	-
	LLMCompiler	77.13	**5.47**	**3.74**×	77.80	**11.83**	**2.82**×
ParallelQA	ReAct	89.09	35.90	1.00×	59.59	15.47	1.00×
	OAI Parallel Function	87.32	19.29	1.86×	-	-	-
	LLMCompiler	89.38	**16.69**	**2.15**×	68.14	**26.20**	**2.27**×
Game of 24	Tree-of-Thoughts	74.00	241.2	1.00×	30.00	952.06	1.00×
	LLMCompiler	75.33	**83.6**	**2.89**×	32.00	**456.02**	**2.09**×

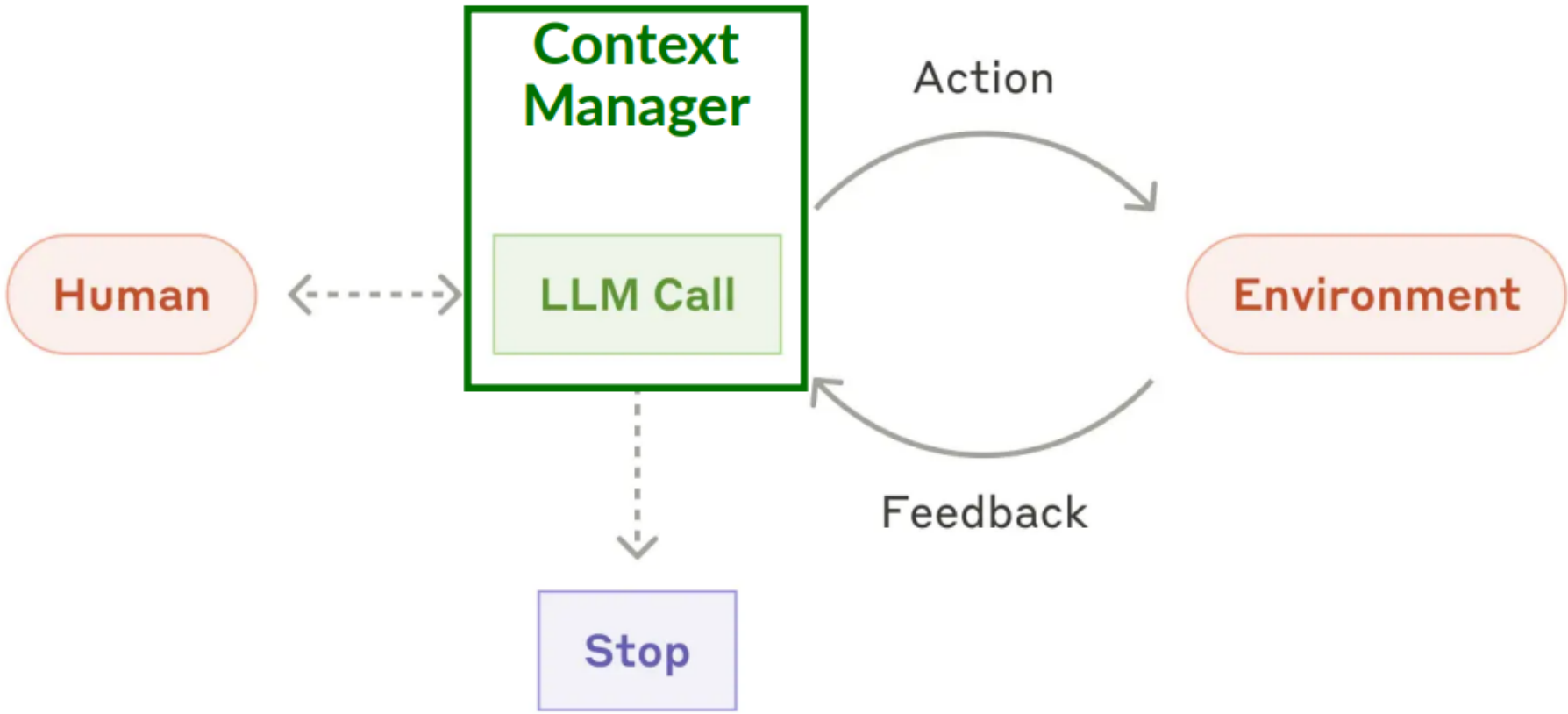
Recap

- **ReAct**: Reasoning before acting is useful.
- **Reflexion**: Self-reflection is incorporated into the reasoning today.
- **CodeAct**: Code is a good form factor for actions.
- **ReWOO/LLMCompiler**: Compiling workflows is primarily done with parallel tool calls today. Planning is still important.
  - Sometimes workflows are still used for “sub-agent” orchestration.

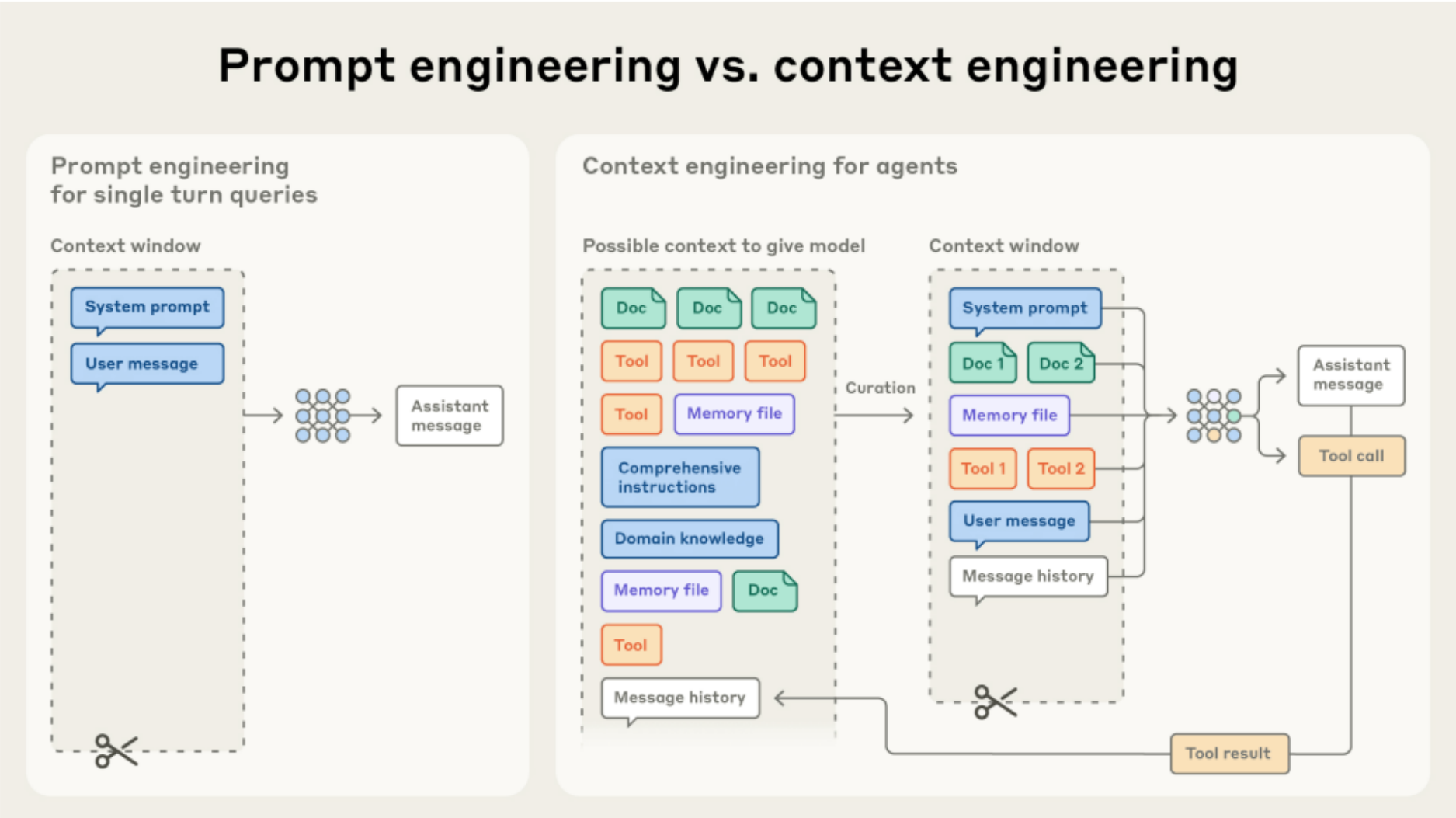


Context Management

Agents: Context

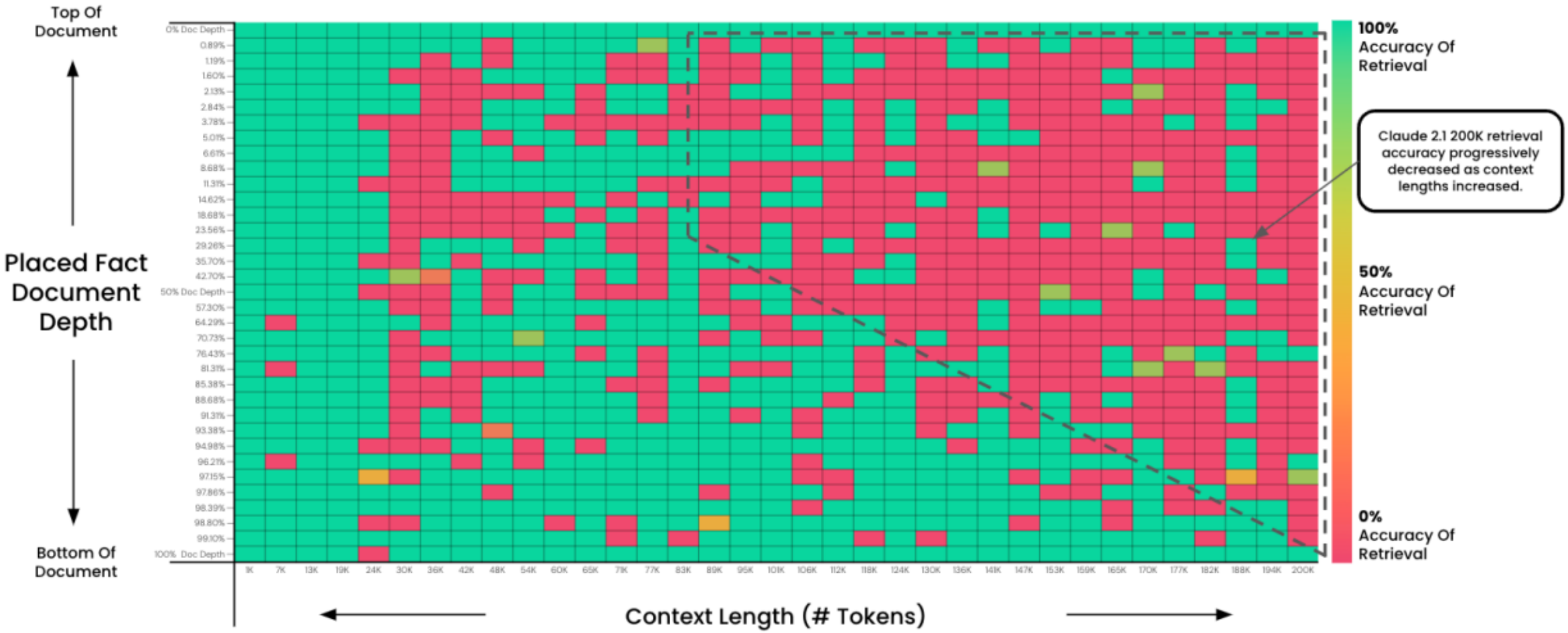


Context Engineering



Pressure Testing Claude-2.1 200K via "Needle In A HayStack"

Asking Claude 2.1 To Do Fact Retrieval Across Context Lengths & Document Depth



Goal: Test Claude 2.1 Ability To Retrieve Information From Large Context Windows

A fact was placed within a document. Claude 2.1 (200K) was then asked to retrieve it. The output was evaluated (with GPT-4) for accuracy.
This test was run at 35 different document depths (top > bottom) and 35 different context lengths (1K >200K tokens).
Document Depths followed a sigmoid distribution

Context Engineering: Principles

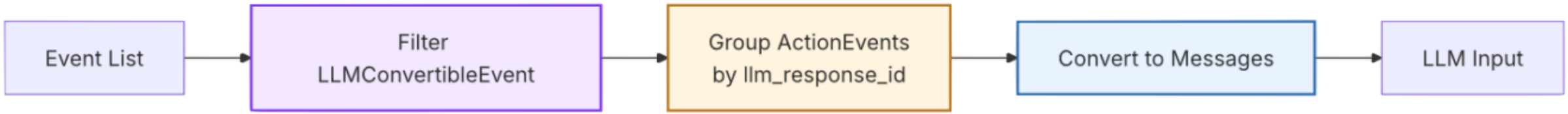
Focus the Context

- Keep only what you need for the task at hand in the context window.
- The entire history does not need to be in the context.
- Excessive information is context-bloat.

Preserve Continuity

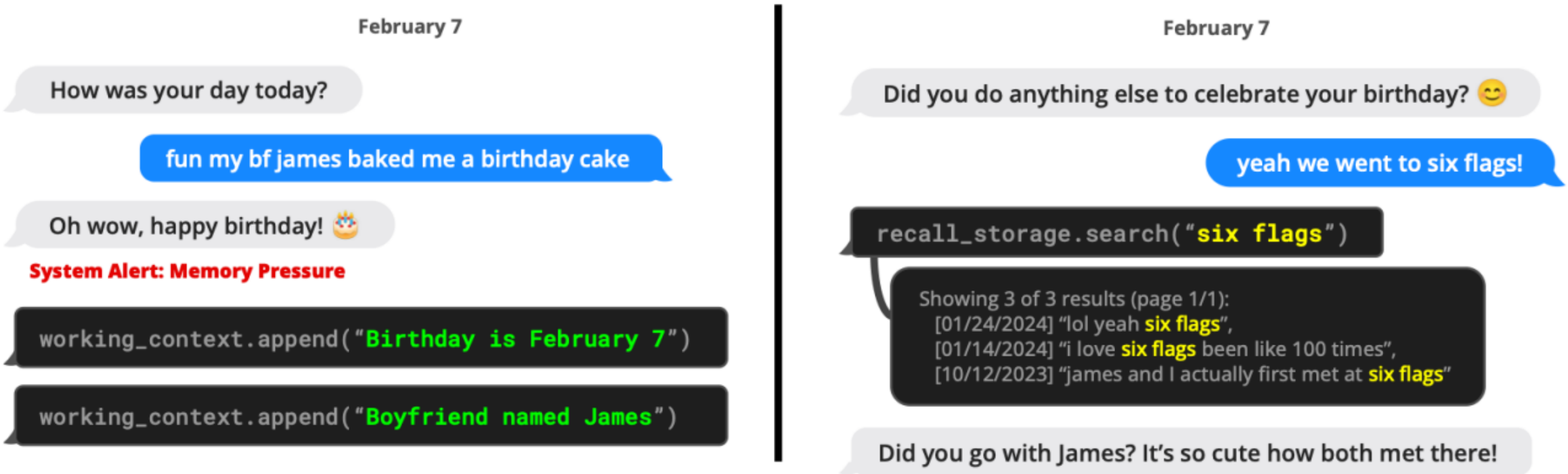
- Manipulate the context to preserve requirements, goals, constraints, progress, etc.
- Practically this also means preserving the KV Cache

Persistent Event Log: OpenHands



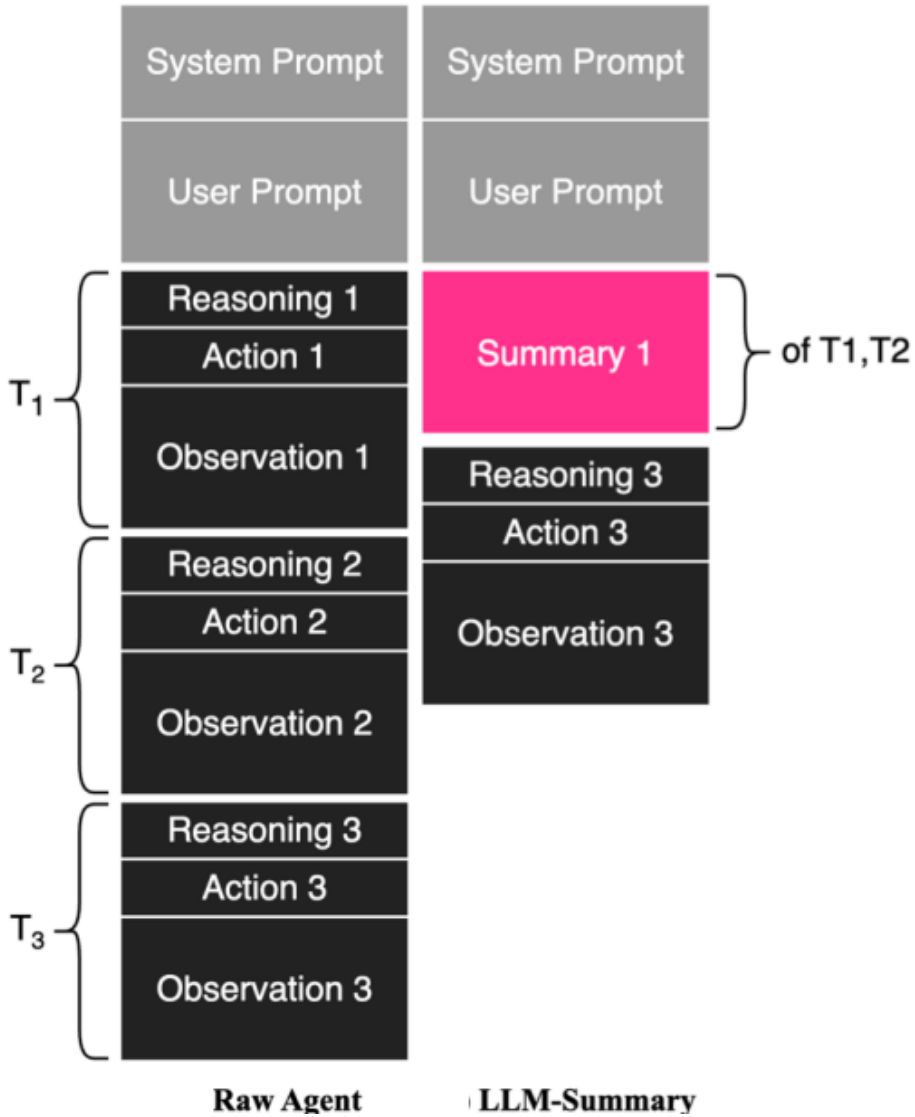
- Everything gets preserved exactly as it happened even if the LLM input must be changed. The event log history can be immutable (append only).
- For every call to the LLM the input is reconstructed from the event log.
- An event log also enables hooks for external observers to monitor the system state.

Memory Offloading: MemGPT



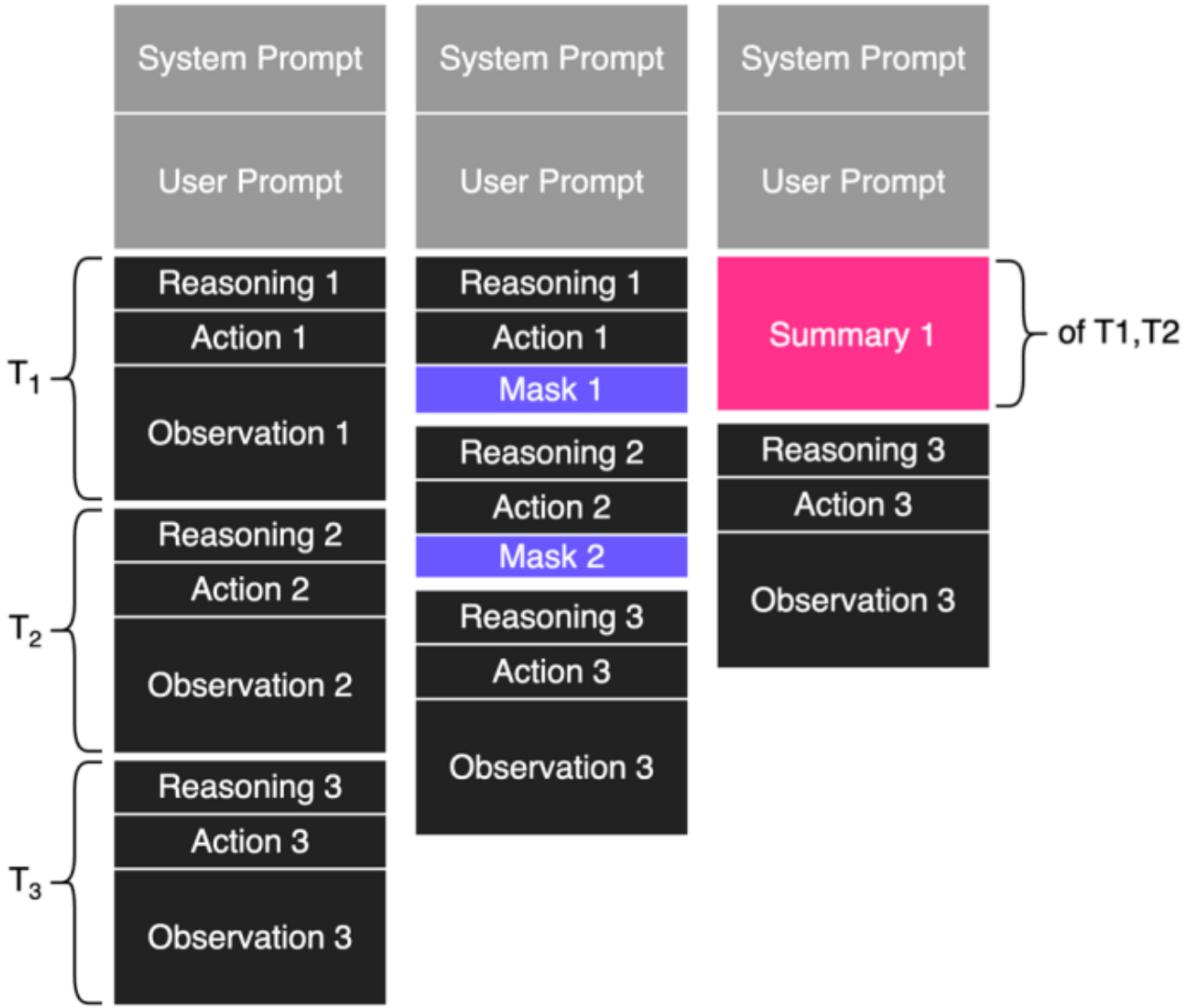
Storing things to memory or files frees up parts of the context to prune

Compaction



- Have the LLM summarize prior reasoning, action, observation instances.
- Compress the history into fewer tokens.
- Can be helpful for the compaction itself to be agentic.

Observation Masking



(a) Raw Agent **(b) Observation Masking** **(c) LLM-Summary**

- Instead of having an LLM compact the context into a summary you can take old observations and truncate them.
- Especially useful for coding agents where the observations can be quite long.

Observation Masking

